McChrystal Overwatch Agent — Technical Guide

> AI-powered business development automation built on Claude Code. One command triggers the full pipeline: discover prospects, build dossiers, draft outreach, track market intelligence — no API keys, no database, no CRM.

- **Live Dashboard**: https://ace1523.github.io/bd-agent/
- **Current Scale**: 35 prospects, 35 dossiers, 35 outreach packages, 13 market sectors

---

Introduction

Overwatch is McChrystal Group's AI-powered business development platform — and it's live now. You can explore the full working dashboard at ace1523.github.io/bd-agent, where every prospect, dossier, outreach package, and market intelligence feed described in this guide is accessible and interactive.

The platform automates McChrystal Group's entire BD pipeline from first signal to final outreach. It starts with **Discovery**: Claude Code searches the web for organizations showing signs they need McChrystal Group — leadership transitions, mergers, organizational restructurings, rapid growth, activist investor pressure, earnings misses, and other trigger events. It scores each prospect against a detailed Ideal Customer Profile covering revenue range, employee count, industry tier, signal recency, and McChrystal-specific fit dimensions. The result is a ranked pipeline of companies with specific, current reasons to engage — not generic Fortune 500 names, but non-obvious targets like PE-backed roll-ups, fresh spinoffs, and niche industry leaders mid-transformation.

From there, the pipeline moves into **Research**. For every scored prospect, Claude Code builds a comprehensive 10-section dossier — the kind of briefing a Senior Partner can read in under 10 minutes and walk into a meeting fully prepared. Dossiers cover the organization snapshot, financial health, leadership team profiles (with priority outreach targets identified), culture and structure signals, recent trigger events, McChrystal fit assessment, conversation entry points, brand insights, and a deep strategic analysis that builds the full case for pursuit. That final section includes fit dimensions with competitor displacement logic, enterprise issues, expected engagement outcomes, a stakeholder map, and a phased land-and-expand strategy with dollar estimates.

Next comes **Outreach**. The platform generates three independent cold email versions (A/B/C) per prospect — trigger-based, insight-based, and warm-angle — each with a genuinely different opening strategy. These aren't sequential follow-ups; they're alternatives a Partner can choose from based on what feels right for the

relationship. Each package also includes a LinkedIn message. Every email is 150–200 words, references something specific to that prospect, and targets a champion-level contact (VP/SVP/Head of Strategy) who feels the pain daily — not the CEO.

Running alongside all of this is **Market Intelligence** — a living feed covering 13 industry sectors (both niche categories like PE-backed roll-ups and defense consolidation, and general markets like financial services and technology). Each sector is refreshed with curated articles, key trends, McChrystal angles, and a "Companies to Watch" list that feeds back into the next discovery cycle.

The key architectural insight: **Claude Code is the entire engine.** There are no API integrations, no database, no CRM, no training pipelines. Claude Code performs live web research, reasons about prospect fit, builds structured Pydantic data models, generates Markdown reports, and pushes everything to a static dashboard hosted on GitHub Pages. The system is a Python library that Claude Code calls directly — every module is designed to be invoked conversationally, not through a CLI or scheduled jobs. A single instruction can trigger the full pipeline end-to-end: discover, score, research, draft outreach, update the dashboard, and deploy.

But there's a deeper pattern at work here: **the filesystem *is* the agent.** Overwatch has no database, no message broker, no container orchestration, no orchestrator process. The entire runtime is three directories. `bd/` is the logic layer — Python modules that define scoring, report generation, data models, and persistence. `data/` is the persistence layer — Markdown reports for human readability and JSON for machine state. `docs/` is the presentation layer — a static single-page application that reads `dashboard.json` and renders everything in the browser. The Unix filesystem provides what most agent frameworks build from scratch: `fcntl.flock` for file-level concurrency control, JSON files for state management, Markdown for human-readable output, and git for versioning, audit trail, and deployment (push to main → GitHub Pages updates automatically).

This inverts the typical AI agent architecture. Most frameworks wrap LLMs in layers of scaffolding — vector databases for retrieval, tool-calling APIs, prompt chaining engines, orchestration layers, and custom infrastructure to manage state between steps. Overwatch doesn't need any of that because Claude Code already has the capabilities baked in: web search, file I/O, code execution, and multi-step reasoning. So instead of building infrastructure *around* the model, Overwatch builds a well-organized project directory that Claude Code navigates *naturally*. The instructions live in `CLAUDE.md`, the logic lives in Python modules Claude Code can import and call, and the output lives in flat files that are both human-readable and machine-parseable. There is no training, no fine-tuning, no prompt chains, no API keys. The system is the filesystem — and the filesystem is the system.

This pattern is reproducible. Anyone can build a domain-specific AI agent the same way: structure a project directory with clear separation between logic, data, and presentation; write Python modules that Claude Code can call to do the domain-specific work; put the workflow instructions in `CLAUDE.md`; and let Claude Code orchestrate. The result is an agent that's auditable (every output is a file in git), debuggable (read the Markdown), and deployable (push to main) — with zero infrastructure to maintain.

This guide is the complete technical reference for how it all works. For a hands-on look at what the platform produces, start with the live dashboard — browse the pipeline, expand a dossier, read the outreach drafts, and explore the market intelligence feed. Then come back here for the architecture, data models, scoring

logic, and implementation details behind it all.

Table of Contents

1. Architecture Overview
2. Installation & Setup
3. McChrystal Group Context
4. Ideal Customer Profile (ICP)
5. Scoring Model
6. Data Models
7. Phase 1 — Discovery
8. Phase 2 — Research (Dossiers)
9. Phase 3 — Outreach
10. Market Intelligence
11. Pipeline Orchestration
12. Persistence & File Locking
13. The Dashboard — Static SPA
14. Deployment & Git Workflow
15. Configuration Reference
16. Future Roadmap
17. Troubleshooting
18. Appendix A — Module API Reference
19. Appendix B — Dashboard app.js Function Index

---

1. Architecture Overview

```
Claude Code (web search + reasoning)
        │
        ▼
  Pydantic Models (bd/models.py)
        │
        ▼
  Report Generators (bd/discover/, bd/research/, bd/outreach/, bd/market/)
        │
        ▼
  Persistence (bd/save.py) ──► Markdown reports (data/*.md)
        │                        + JSON (data/dashboard.json)
        │
        ▼
```

```
Dual-write ──► docs/dashboard.json
      │
      ▼
Static SPA (docs/index.html + app.js + style.css)
      │
      ▼
GitHub Pages (auto-deploy on push to main)
```

Three layers:

Layer	Files	Purpose
Data	`bd/models.py`	Pydantic model definitions for all entities
Logic	`bd/discover/`, `bd/research/`, `bd/outreach/`, `bd/market/`, `bd/save.py`, `bd/pipeline.py`	Scoring, report generation, persistence, orchestration
Presentation	`docs/index.html`, `docs/app.js`, `docs/style.css`	Static SPA served via GitHub Pages

Key design decisions: - No server, no database — everything is files (Markdown + JSON) - No API keys — Claude Code has built-in web search - File locking (`fcntl.flock`) for parallel subagent concurrency - Dual-write ensures `data/dashboard.json` and `docs/dashboard.json` stay in sync

Directory Structure

```
bd-agent/
├── bd/                          # Python package
│   ├── __init__.py
│   ├── config.py                # ICP thresholds, scoring weights, search config
│   ├── dashboard.py             # JSON export + Markdown bootstrap parsers
│   ├── formatting.py            # Revenue/employee display helpers
│   ├── models.py                # All Pydantic models
│   ├── pipeline.py              # Orchestration helpers
│   ├── save.py                  # Persistence with file locking
│   ├── discover/
│   │   ├── __init__.py
│   │   ├── report.py            # Discovery report generator
│   │   └── scorer.py            # ICP scoring (4 components)
│   ├── market/
│   │   ├── __init__.py
```

```
│   │   └── report.py            # Market intelligence report generator
│   ├── outreach/
│   │   ├── __init__.py
│   │   └── drafter.py           # Outreach report generator
│   └── research/
│       ├── __init__.py
│       └── report.py            # Dossier report generator
├── data/                        # Generated outputs
│   ├── dashboard.json           # Canonical JSON (all pipeline data)
│   ├── discovery.md             # Appended discovery reports
│   ├── research.md              # Appended dossier reports
│   ├── outreach.md              # Appended outreach reports
│   ├── market_intelligence.md   # Replaced on each refresh
│   └── proposals/               # Future: reference proposals for Phase 4
├── docs/                        # Static SPA (GitHub Pages root)
│   ├── index.html               # SPA shell (58 lines)
│   ├── app.js                   # Client-side application (2,279 lines)
│   ├── style.css                # Responsive styling (1,557 lines)
│   └── dashboard.json           # Copy of data/dashboard.json (auto-synced)
├── populate_markets.py          # Market intelligence CLI + seed data
├── pyproject.toml               # Package config
├── CLAUDE.md                    # Agent instructions (Claude Code reads this)
└── TECHNICAL_GUIDE.md           # This file
```

---

2. Installation & Setup

Prerequisites: Python 3.11+, Git, Claude Code CLI

```
git clone https://github.com/Ace1523/bd-agent.git
cd bd-agent
python3 -m venv .venv
source .venv/bin/activate
pip install -e .
```

Dependencies (from `pyproject.toml`): - `pydantic>=2.5` — data models and validation - `rich>=13.7` — terminal formatting

No API keys, no `.env` file. Claude Code provides web search natively.

Verify installation:

```
python -c "from bd.models import Prospect; print('OK')"
```

3. McChrystal Group Context

McChrystal Group is a leadership advisory firm founded by General Stanley McChrystal. Core thesis: organizations operate in VUCA (volatile, uncertain, complex, ambiguous) environments requiring the same decentralized authority and shared consciousness that transformed Joint Special Operations Command in Iraq.

Core offerings: - Leadership development and executive coaching - Organizational design and transformation - Team of Teams operating model implementation - Cross-functional alignment and adaptability consulting - Crisis leadership and high-stakes decision-making - Culture change and trust-building at scale

Why this matters for scoring: Every scoring decision, fit assessment, and outreach angle is calibrated to MG's positioning vs. McKinsey (strategy), Deloitte (implementation), and Korn Ferry (talent). MG's differentiator is the *operating model* — they solve the organizational dysfunction that prevents strategy from executing.

---

4. Ideal Customer Profile (ICP)

Firmographics

Parameter	Value	Notes
Revenue	$500M–$10B	Sweet spot; premium engagements $500K–$5M+
Employees	500+ required	2,000+ preferred

Signal Types

9 signal types classify organizational moments that indicate MG fit:

Signal Type	Enum Value	Definition
Reorg	`reorg`	Restructuring, downsizing, organizational redesign
M&A	`m_and_a`	Mergers, acquisitions, divestitures, go-private deals

Signal Type	Enum Value	Definition
Leadership Change	`leadership_change`	C-suite departures, new CEO/COO/CTO appointments
Earnings Miss	`earnings_miss`	Revenue shortfall, guidance cuts, EBITDA pressure
Rapid Growth	`rapid_growth`	Revenue surge, rapid scaling, market expansion
Transformation	`transformation`	Digital transformation, business model pivot, strategic overhaul
Hiring Surge	`hiring_surge`	Aggressive hiring, talent acquisition push, new division buildout
Funding	`funding`	IPO, capital raise, PE investment, debt restructuring
Partnership	`partnership`	Strategic alliances, joint ventures, platform partnerships

Each signal type has 10-16 associated keywords in `SIGNAL_KEYWORDS` (see Configuration Reference).

Industry Tiers

Tier	Industries
Tier 1 (proven)	Defense, Aerospace, Federal Government, Financial Services, Healthcare, Industrial, Manufacturing
Tier 2 (adjacent)	Technology, Private Equity, Energy, Logistics, Supply Chain, Professional Services
Tier 3 (unconventional)	Sports, Media, Entertainment, Nonprofit, NGO, Education, Agriculture, Law Enforcement, First Responders, Infrastructure, Construction

Fit Ratings

Rating	Criteria
Strong	Problem maps to MG capability AND 2+ of: active trigger (last 6 months), clear buyer, budget exists (revenue >$1B), low competitive barrier, organizational urgency

Rating	Criteria
Moderate	Clear fit but uncertain: financial distress, no entry point, old trigger, incumbent competitor, cultural resistance
Speculative	Plausible fit, significant unknowns: inferred problem, Tier 3 industry, size/budget below threshold

---

5. Scoring Model

Total: 0–100 points across four components.

Component	Max	What It Measures
Revenue	25	Fit within $500M–$10B sweet spot
Employees	20	2,000+ preferred, 500+ minimum
Signals	35	Top 4 signals + recency bonus
Fit	20	High-fit signal types + signal diversity

Revenue Scoring (0–25)

```python
def score_revenue(prospect: Prospect) -> float:
    """Score revenue fit (0–25). Sweet spot is $500M–$10B."""
    rev = prospect.revenue_estimate
    if rev is None:
        return SCORE_REVENUE_MAX * 0.3  # 7.5 pts for unknown

    if ICP_REVENUE_MIN <= rev <= ICP_REVENUE_MAX:
        # Full marks in the sweet spot; peak score in middle of range
        midpoint = (ICP_REVENUE_MIN + ICP_REVENUE_MAX) / 2
        distance = abs(rev - midpoint) / midpoint
        return SCORE_REVENUE_MAX * max(0.7, 1.0 - distance * 0.3)

    if rev < ICP_REVENUE_MIN:
        ratio = rev / ICP_REVENUE_MIN
        return SCORE_REVENUE_MAX * ratio * 0.5

    # Over $10B — still a fit, just slightly less ideal
    return SCORE_REVENUE_MAX * 0.6
```

Curve behavior: - $5.25B (midpoint) → 25 pts (100%) - $500M or $10B (edges) → 17.5 pts (70%) - $250M → 6.25 pts (ratio × 50%) - $15B → 15 pts (60%) - Unknown → 7.5 pts (30%)

Employee Scoring (0–20)

```
def score_employees(prospect: Prospect) -> float:
    """Score employee count fit (0–20). 500+ required, 2K+ preferred."""
    count = prospect.employee_count
    if count is None:
        return SCORE_EMPLOYEE_MAX * 0.3  # 6.0 pts

    if count < ICP_EMPLOYEE_MIN:  # < 500
        ratio = count / ICP_EMPLOYEE_MIN
        return SCORE_EMPLOYEE_MAX * ratio * 0.3

    if count < ICP_EMPLOYEE_PREFERRED:  # 500–2,000
        ratio = (count - ICP_EMPLOYEE_MIN) / (ICP_EMPLOYEE_PREFERRED -
        ICP_EMPLOYEE_MIN)
        return SCORE_EMPLOYEE_MAX * (0.5 + ratio * 0.2)

    # 2000+ — full credit with diminishing returns
    bonus = min((count - ICP_EMPLOYEE_PREFERRED) / 50_000, 1.0) * 0.3
    return SCORE_EMPLOYEE_MAX * min(0.7 + bonus, 1.0)
```

Signal Scoring (0–35)

```
def score_signals(prospect: Prospect) -> float:
    """Score signal strength (0–35). More signals and more recent = higher."""
    if not prospect.signals:
        return 0.0

    today = date.today()
    total = 0.0
    max_per_signal = SCORE_SIGNAL_MAX / 3  # ~11.67 per signal

    for i, signal in enumerate(prospect.signals[:4]):  # Cap at top 4
        base = max_per_signal * 0.6  # ~7.0 base

        # Recency bonus
        if signal.date:
            days_ago = (today - signal.date).days
            if days_ago < 30:
                base += max_per_signal * 0.4   # +4.67
            elif days_ago < 90:
                base += max_per_signal * 0.3   # +3.50
            elif days_ago < 180:
                base += max_per_signal * 0.2   # +2.33
```

```
            else:
                base += max_per_signal * 0.1   # +1.17
        else:
            base += max_per_signal * 0.15      # +1.75 unknown

        total += base

    return min(total, SCORE_SIGNAL_MAX)
```

Fit Scoring (0–20)

```
HIGH_FIT_SIGNALS = {
    SignalType.reorg, SignalType.transformation, SignalType.m_and_a,
    SignalType.hiring_surge, SignalType.funding, SignalType.partnership,
}

def score_fit(prospect: Prospect) -> float:
    """Score McChrystal Group fit (0–20)."""
    if not prospect.signals:
        return 0.0

    high_fit_count = sum(1 for s in prospect.signals if s.type in HIGH_FIT_SIGNALS)
    signal_diversity = len({s.type for s in prospect.signals})

    fit = 0.0
    fit += min(high_fit_count * 5, 12)    # 5 pts per high-fit signal, max 12
    fit += min(signal_diversity * 2, 8)    # 2 pts per unique type, max 8

    return min(fit, SCORE_FIT_MAX)
```

Worked Example

Prospect: Acme Corp — $2B revenue, 5,000 employees, 3 signals (M&A 15 days ago, reorg 45 days ago, leadership_change 200 days ago)

Component	Calculation	Score
Revenue	$2B in sweet spot, distance from $5.25B midpoint = 0.619 → max(0.7, 1.0 - 0.619×0.3) = max(0.7, 0.814) = 0.814 → 25 × 0.814	**20.4**
Employees	5,000 > 2,000 → bonus = min(3000/50000, 1.0)×0.3 = 0.018 → 20 × min(0.718, 1.0)	**14.4**

Component	Calculation	Score
Signals	Signal 1 (M&A, 15d): 7.0 + 4.67 = 11.67. Signal 2 (reorg, 45d): 7.0 + 3.50 = 10.50. Signal 3 (leadership, 200d): 7.0 + 1.17 = 8.17. Total = 30.3, capped at 35	**30.3**
Fit	High-fit: M&A + reorg = 2 × 5 = 10. Diversity: 3 types × 2 = 6. Total = 16	**16.0**
Total		**81.1**

---

6. Data Models

All models live in `bd/models.py` using Pydantic v2.

Discovery Models

```
class SignalType(str, Enum):
    reorg = "reorg"
    m_and_a = "m_and_a"
    leadership_change = "leadership_change"
    earnings_miss = "earnings_miss"
    rapid_growth = "rapid_growth"
    transformation = "transformation"
    hiring_surge = "hiring_surge"
    funding = "funding"
    partnership = "partnership"

class FitTier(str, Enum):
    tier_1 = "tier_1"  # Conventional
    tier_2 = "tier_2"  # Adjacent
    tier_3 = "tier_3"  # Unconventional

class Signal(BaseModel):
    type: SignalType
    description: str
    date: Union[datetime.date, None] = None
    source: Union[str, None] = None

class Prospect(BaseModel):
    company_name: str
    revenue_estimate: Union[float, None] = None
    employee_count: Union[int, None] = None
```

```
    industry: Union[str, None] = None
    tier: Union[FitTier, None] = None
    signals: list[Signal] = []
    score: float = 0.0
    source_urls: list[str] = []
    company_overview: Union[str, None] = None
    summary: Union[str, None] = None
    entry_point: Union[str, None] = None
    conversation_hook: Union[str, None] = None
```

Research Models

```
class Contact(BaseModel):
    name: str
    title: str
    tenure: Union[str, None] = None
    background: Union[str, None] = None
    linkedin: Union[str, None] = None
    notes: Union[str, None] = None
    is_priority_target: bool = False
    priority_rationale: Union[str, None] = None

class FitRating(str, Enum):
    strong = "strong"
    moderate = "moderate"
    speculative = "speculative"

class TriggerEvent(BaseModel):
    date: Union[datetime.date, None] = None
    event: str
    relevance: str
    source: Union[str, None] = None

class FitAssessment(BaseModel):
    primary_problem: str
    best_capability_fit: str
    likely_objections: list[str] = []
    competitive_landscape: list[str] = []
    rating: FitRating = FitRating.moderate
    rating_rationale: Union[str, None] = None

class ConversationEntry(BaseModel):
    opening_question: str
    framing: Union[str, None] = None

class Dossier(BaseModel):
    prospect: Prospect
    # Sec 1: Organization Snapshot
```

```
    legal_name: Union[str, None] = None
    headquarters: Union[str, None] = None
    founded_year: Union[int, None] = None
    ownership_structure: Union[str, None] = None
    geographic_footprint: Union[str, None] = None
    # Sec 2: Financial Health
    financial_health: Union[str, None] = None
    # Sec 3: Leadership
    key_contacts: list[Contact] = []
    # Sec 4: Culture & Structure
    org_structure: Union[str, None] = None
    culture_signals: Union[str, None] = None
    # Sec 5: Trigger Events
    trigger_events: list[TriggerEvent] = []
    # Sec 6: Fit Assessment
    fit_assessment: Union[FitAssessment, None] = None
    # Sec 7: Conversation Entry Points
    conversation_entries: list[ConversationEntry] = []
    recommended_meeting_framing: Union[str, None] = None
    # Sec 8: Brand Insights
    brand_insights: Union[str, None] = None
    # Sec 9: Deep Fit Analysis
    deep_fit_analysis: Union[str, None] = None
    # Legacy fields
    pain_points: list[str] = []
    competitors_present: list[str] = []
    transformation_timeline: Union[str, None] = None
    budget_context: Union[str, None] = None
    detailed_analysis: str = ""
```

Outreach Models

```
class ColdEmail(BaseModel):
    version: str       # "A", "B", or "C"
    label: str         # "Trigger-based", "Insight-based", "Warm angle"
    subject: str
    body: str
    hook: str

class LinkedInMessage(BaseModel):
    message_type: str  # "connection_request" or "inmail"
    subject: Union[str, None] = None
    body: str
    hook: str

class OutreachPackage(BaseModel):
    dossier: Dossier
    target_contact: Contact
```

```
    cold_emails: list[ColdEmail] = []
    linkedin_message: Union[LinkedInMessage, None] = None
```

Market Intelligence Models

```
class SectorCategory(str, Enum):
    niche = "niche"
    general = "general"

class MarketArticle(BaseModel):
    title: str
    url: str
    source: str
    date: Union[datetime.date, None] = None
    summary: str
    relevance_note: Union[str, None] = None

class MarketSector(BaseModel):
    name: str
    category: SectorCategory
    overview: str
    key_trends: list[str] = []
    mcchrystal_angle: str
    articles: list[MarketArticle] = []
    last_refreshed: Union[datetime.date, None] = None
    prospect_candidates: list[str] = []
```

---

7. Phase 1 — Discovery

Flow: Web research → Prospect objects → `score_prospect()` → `generate_report()` → save

Step 0: Mine Market Intelligence

Before fresh research, scan existing `market_intelligence` in `dashboard.json` for companies mentioned in articles showing ICP signals. Cross-reference against existing prospects to avoid duplicates.

Research Quality Standards

- Prioritize non-obvious companies — no household names (Boeing, Amazon, Google)
- Every prospect must have a specific, *current* reason to reach out
- Verify with multiple sources, confirm revenue/employee data
- Specific dates (month/year minimum), not “recently”

- Revenue and employee figures must be sourced or marked `[INFERRED]`

Report Generation

`generate_report()` in `bd/discover/report.py`: 1. Renders header with date and count 2. Summary table (company, tier, score, revenue, employees, signals) 3. Detail sections grouped by tier (Tier 1 → Tier 2 → Tier 3) 4. Each prospect: revenue, employees, industry, tier, signals with dates/sources, entry point, conversation hook, summary 5. Calls `save_discovery()` which appends to `data/discovery.md` and updates `dashboard.json`

Deduplication

`get_existing_prospects()` in `bd/pipeline.py` loads all prospects from `dashboard.json`, keeping the highest-scoring entry per company name.

---

8. Phase 2 — Research (Dossiers)

Comprehensive dossiers a Senior Partner can read in under 10 minutes. **10 sections:**

#	Section	Key Content
1	Organization Snapshot	Legal name, HQ, founded year, ownership, geography
1b	Company Overview	3-5 sentence narrative (standalone introduction)
2	Financial Health & Growth Stage	Revenue trajectory, profitability, analyst sentiment
3	Leadership Team Profiles	C-suite + board; priority targets are champion-level (VP/SVP), NOT CEO
4	Culture & Structure Signals	Hierarchy vs. flat, Glassdoor patterns, transformation programs
5	Recent News & Trigger Events	5-7 developments (last 18 months) with dates and relevance
6	McChrystal Fit Assessment	Primary problem, capability fit, objections, competitors, rating
7	Conversation Entry Points	2-3 opening questions, recommended meeting framing
8	Brand Insights & Market Positioning	Brand value, identity evolution, CMO strategy, brand threats

#	Section	Key Content
9	Deep McChrystal Fit Analysis	The critical section — 6 subsections (see below)

Section 9 Deep Dive

Subsection	Content
9a. Fit Dimensions	3-6 reinforcing dimensions with competitor displacement logic
9b. Cumulative Case	Synthesized thesis, reinforcing signal chain, revenue potential estimate
9c. Enterprise Issues	5-8 detailed organizational problems with MG capability mapping
9d. Expected Outcomes	5-7 measurable deliverables for a proposal
9e. Key Stakeholders	Table: Business Unit, Leader, Function, MG Relevance
9f. Opportunity Thesis	Signal convergence, structural paradox, phased engagement plan, competitive displacement, multi-threaded pursuit map

Execution

- Split prospects into batches of 3-4
- Launch 2-3 parallel subagents (or build directly if subagents lack Bash permission)
- Each subagent: deep web research → build `Dossier` object → `generate_dossier_report()`
- Report calls `save_research()` which appends to `data/research.md` + updates JSON

---

9. Phase 3 — Outreach

3 independent cold email versions (A/B/C) per prospect — **not sequential follow-ups**.

Version	Strategy	Lead With
A	Trigger-based	Specific recent event from dossier
B	Insight-based	Industry trend → pivot to prospect's situation

Version	Strategy	Lead With
C	Warm angle	Human connection (shared network, military background, etc.)

Email Structure (all versions)

1. **Opening hook** — specific, relevant, human (2-3 sentences)
2. **Bridge** — why McChrystal is reaching out now (2-3 sentences)
3. **Credibility signal** — one concrete proof point (1-2 sentences)
4. **Soft ask** — 20-minute call, low-friction (1-2 sentences)
5. **Signature block**

Constraints: 150-200 words max, no buzzwords, no competitor names, not AI-detectable, `[GAP]` flag if no real angle exists.

LinkedIn Message

- 50-100 words, casual tone
- `connection_request` (300 char limit, no subject) or `inmail` (has subject)
- One specific reference to the contact or company

Targeting

Champion-level contacts (VP, SVP, Chief Transformation Officer, Head of Strategy) — NOT the CEO. CEO is marked "executive sponsor."

Execution

- Split into batches of 5 per subagent (2 parallel)
- Loads dossier data from `dashboard.json` + fresh web research for hooks
- Builds `OutreachPackage` → `generate_outreach_report()` → `save_outreach()`

---

10. Market Intelligence

13 sectors organized in two categories:

Niche (8): PE-Backed Roll-Ups, Post-Merger Integrations, Defense Consolidation, Healthcare M&A, Energy Transition, Tech Spinoffs & Carve-outs, Government Transformation, Sports & Entertainment Ownership

General (5): Technology & AI, Financial Services, Industrial & Manufacturing, Healthcare Systems, Energy & Utilities

Refresh Workflow

```
# Check freshness of all sectors
python3 populate_markets.py --status
```

1. Claude Code performs web searches per sector (last 30 days)
2. Builds `MarketSector` objects with 5-10 articles each
3. Calls `generate_market_report(sectors)` — saves Markdown + **replaces** (not appends) `market_intelligence` in dashboard.json
4. Push to GitHub to deploy

Freshness Badges

Badge	Threshold
Green	≤ 7 days
Amber	≤ 30 days
Gray	> 30 days (stale)

Cross-Pollination

Each sector's `prospect_candidates` field captures companies showing ICP signals. Before discovery, mine these candidates and cross-reference against existing pipeline.

Search Query Templates

Four categories of search queries per sector (from `config.py`): - **Signal Scan** — restructuring, merger, acquisition, new CEO, transformation - **Leadership Moves** — CEO appointed/departed, board shakeups, activist investors - **Macro Shifts** — industry trends, regulatory change, AI disruption, tariffs - **Org Challenges** — organizational challenges, integration, growing pains, talent gap

---

11. Pipeline Orchestration

Three helpers in `bd/pipeline.py`:

`get_existing_prospects()`

Reads all unique prospects from `dashboard.json`. Deduplicates by company name, keeping the highest score.

```python
def get_existing_prospects() -> list[Prospect]:
    data = _load_dashboard_json()
    best: dict[str, dict] = {}
    for run in data.get("discovery_runs", []):
        for p in run.get("prospects", []):
            name = p.get("company_name", "")
            existing = best.get(name)
            if not existing or p.get("score", 0) > existing.get("score", 0):
                best[name] = p
    return [Prospect.model_validate(v) for v in best.values()]
```

`pipeline_status()`

Returns counts and gap analysis:

```python
{
    "prospects": 35,
    "dossiers": 35,
    "outreach_packages": 35,
    "market_sectors": 13,
    "missing_dossiers": [],      # prospects without dossiers
    "missing_outreach": [],      # dossiers without outreach
}
```

`clear_phase(phase)`

Resets a phase (`'discovery'` , `'research'` , `'outreach'` , or `'market'`): clears the Markdown file and removes the corresponding array from `dashboard.json` .

Full Execution Strategy

Step	Action
0	Mine market intelligence for prospect candidates
1	Discover — web research, build Prospects, score, generate report
2	Research — parallel subagents build Dossiers (batches of 3-4)
3	Outreach — parallel subagents build OutreachPackages (batches of 5)
4	Verify — `pipeline_status()` confirms completeness

Step	Action
5	Deploy — push to GitHub

---

12. Persistence & File Locking

Save Functions

Function	File	Semantics
`save_discovery()`	`data/discovery.md`	Append
`save_research()`	`data/research.md`	Append
`save_outreach()`	`data/outreach.md`	Append
`save_market_intelligence()`	`data/market_intelligence.md`	Replace
`clear_outreach()`	Both files	Reset
`clear_market_intelligence()`	Both files	Reset

Dual-Write

Every save writes to both `data/dashboard.json` and `docs/dashboard.json`:

```
def _save_dashboard_json(data: dict) -> None:
    DATA_DIR.mkdir(exist_ok=True)
    content = json.dumps(data, indent=2, default=str)
    DASHBOARD_JSON.write_text(content)
    if SITE_DIR.exists():
        (SITE_DIR / "dashboard.json").write_text(content)
```

File Locking

Parallel subagents can write simultaneously. `_locked_update()` uses `fcntl.flock` to prevent race conditions:

```
def _locked_update(update_fn) -> None:
    DATA_DIR.mkdir(exist_ok=True)
    with open(LOCK_FILE, "w") as lock:
```

```
        fcntl.flock(lock, fcntl.LOCK_EX)
        try:
            data = _load_dashboard_json()
            update_fn(data)
            _save_dashboard_json(data)
        finally:
            fcntl.flock(lock, fcntl.LOCK_UN)
```

Lock file: `data/.dashboard.lock`

---

13. The Dashboard — Static SPA

Technology: Vanilla HTML/CSS/JS — no frameworks, no build step.

- `docs/index.html` — 58-line shell with nav, view containers, sidebar, toast
- `docs/app.js` — 2,279 lines: data loading, 8+ view renderers, search, filtering
- `docs/style.css` — 1,557 lines: responsive layout, McChrystal branding

Data Flow

```
Page load → fetch("dashboard.json") → dashboardData (global) → renderAll()
```

Views

View	Description
Home	Pipeline summary KPIs, core capabilities, ICP overview, industry tiers, scoring model
Pipeline	Prospect cards grouped by industry category, filter pills, sorted by score
Research	Dossier cards with full 10-section detail on expand
Outreach	Outreach packages with 3 email versions, copy-to-clipboard
Markets (sidebar)	Right-side panel: 10 most recent articles, “Explore All Markets” link
Markets (full page)	Sector cards grouped by category, articles, freshness badges, prospect candidates
Proposals	Coming soon placeholder

View	Description
How It Works	Pipeline workflow, scoring model, signal types, collapsible prompt blocks
Future	Roadmap items

Industry Category Mapping

`categoryOf()` maps the `industry` field to 7 display categories via keyword matching:

Category	Keywords
Healthcare	health, dental, pharma, drug
Defense & Government	defense, government, federal, aerospace
Technology & Fintech	tech, fintech, software, semiconductor, gaming, telecom, optical, network
Industrial & Logistics	industrial, logistics, auto, chemical, material, parts, collision, fleet, manufacturing, supply chain
Energy	energy, oil, gas, utilit, renewable, power
Home & Business Services	home, service, hvac, plumbing, residential
Media & Consumer	media, entertainment, consumer, retail, apparel, sport, talent

UI Features

- **Filter pills** — "All" groups by category with subheaders; clicking a category filters
- **Company search** — searches across all views, clicking navigates and auto-expands
- **Card expand/collapse** — click to reveal detail
- **Copy-to-clipboard** — on outreach email bodies
- **Scroll-to-top** — appears after 300px scroll
- **Responsive** — hamburger menu on mobile

Color Palette

- McChrystal orange: `#da6123` (brand accents)
- Score green: `score >= 75`
- Score amber: `score >= 60`
- Score gray: `score < 60`

---

14. Deployment & Git Workflow

GitHub Pages serves from `docs/` on the `main` branch.

```
Generate data → save writes to data/ + docs/ → git add → commit → push → auto-deploy
```

No CI/CD pipeline, no build step. Push to main = deploy.

---

15. Configuration Reference

All constants from `bd/config.py`:

ICP Thresholds

Constant	Value
`ICP_REVENUE_MIN`	$500,000,000
`ICP_REVENUE_MAX`	$10,000,000,000
`ICP_EMPLOYEE_MIN`	500
`ICP_EMPLOYEE_PREFERRED`	2,000

Scoring Weights

Constant	Value
`SCORE_REVENUE_MAX`	25
`SCORE_EMPLOYEE_MAX`	20
`SCORE_SIGNAL_MAX`	35
`SCORE_FIT_MAX`	20

Signal Keywords

Signal Type	Keywords
`reorg`	restructuring, reorganization, layoffs, downsizing, workforce reduction, unit consolidation, operating model redesign, cost-cutting, efficiency program, streamlining operations, organizational redesign, flattening the org
`m_and_a`	merger, acquisition, divestiture, go-private, carve-out, spinoff, spin-off, sale of, acquires, to acquire, takeover, combined entity, integration, post-merger, definitive agreement, deal closes
`leadership_change`	new CEO, CEO appointed, CEO departure, CEO transition, C-suite shakeup, new CFO, new COO, new CTO, new CHRO, board shakeup, executive turnover, leadership transition, interim CEO, CEO search, successor named, steps down, resigns, fired, ousted
`earnings_miss`	earnings miss, revenue shortfall, guidance cut, profit warning, missed estimates, below expectations, EBITDA pressure, activist investor, shareholder pressure, proxy fight, stock decline, downgrade, underperformance
`rapid_growth`	rapid growth, revenue surge, scaling, hypergrowth, market expansion, international expansion, new markets, doubling revenue, fastest-growing, growth trajectory, expanding operations, opening new locations
`transformation`	digital transformation, business model pivot, strategic overhaul, modernization, transformation initiative, cloud migration, operating model change, agile transformation, innovation lab, strategic pivot, reinvention, turnaround, transformation officer
`hiring_surge`	hiring spree, talent acquisition, executive hiring, new C-suite roles, leadership team buildout, chief transformation officer, head of strategy, VP hired, building out team,

Signal Type	Keywords
	recruiting push, new division, headcount growth
`funding`	IPO, capital raise, PE investment, private equity, funding round, Series C, Series D, recapitalization, debt restructuring, credit facility, secondary offering, SPAC, goes public, take-private, leveraged buyout
`partnership`	strategic partnership, joint venture, alliance, platform partnership, collaboration agreement, strategic alliance, co-development, licensing deal, channel partnership, ecosystem partner

Capability Keywords

Category	Keywords
`cross_functional_alignment`	silos, siloed, cross-functional, collaboration breakdown, information sharing, lack of coordination, misalignment, organizational friction, handoff failures, interdepartmental
`leadership_development`	leadership crisis, leadership gap, executive development, leadership pipeline, succession planning, coaching, leader readiness, emerging leaders, talent bench
`culture_change`	toxic culture, culture overhaul, employee engagement, Glassdoor reviews, workplace culture, trust deficit, morale, attrition, retention crisis, culture transformation, psychological safety, employee satisfaction
`operating_model`	operating model, org design, organizational effectiveness, team of teams, decentralized, empowered execution, shared consciousness, agility, decision-making speed, bureaucracy, command and control
`crisis_leadership`	crisis management, high-stakes, incident response, crisis leadership, under fire, public scrutiny, regulatory action, congressional hearing, investigation, scandal, safety incident, data breach

Category	Keywords
`post_merger_integration`	integration challenges, merger integration, cultural clash, combined entity, Day 1 readiness, synergy realization, integration PMO, two cultures, integration failure
`scale_challenges`	growing pains, scaling challenges, what got us here, founder transition, startup to enterprise, professionalizing, process maturity, outgrowing, complexity

Source Tiers

Tier	Sources
Tier 1 (preferred)	Wall Street Journal, Reuters, Bloomberg, Financial Times, New York Times, Associated Press
Tier 2	CNBC, Forbes, Fortune, Business Insider, The Information, Axios, Politico, Defense News, Modern Healthcare, S&P Global, PitchBook, PE Hub, Mergermarket
Tier 3 (acceptable)	Industry trade publications, Company press releases, Regional business journals, Seeking Alpha, Yahoo Finance

Article Criteria

Must have one of: specific company named, specific event with date, leadership/deal/org shift reference, data points (revenue, headcount, deal size)

Exclude: pure opinion, unvalidated press releases, articles > 45 days old, duplicates, paywalled with insufficient summary, routine announcements

Prefer: companies in $500M-$10B range, multi-company trend pieces, investigative pieces on org dysfunction, PE-backed/roll-up/post-merger coverage, companies not already in pipeline

Freshness Thresholds

Constant	Value
`FRESHNESS_GREEN_DAYS`	7
`FRESHNESS_AMBER_DAYS`	30

---

16. Future Roadmap

Feature	Description
Phase 4 — Proposals Engine	AI-drafted proposals from dossier data + reference proposals in `data/proposals/`. Needs past proposals, pricing guidelines, win/loss context
LinkedIn Enrichment	Separate project (`~/Desktop/ATL_BD`) enriches regional targeting with MG partner LinkedIn connections — planned for Overwatch integration
Signal-Triggered Discovery	Auto-discover when market intel surfaces high-signal companies
Automated Dossier Enrichment	Re-score and update dossiers as new signals emerge
Contact Discovery	Apollo.io integration for email/phone lookup
Adaptive Opportunity Scoring	Scoring model learns from win/loss feedback
Pipeline Learning & Feedback	Track what outreach works and refine templates
Strategic BD Intelligence Layer	Cross-prospect pattern analysis
Opportunity Network Mapping	Map connections between prospects, board members, investors

---

17. Troubleshooting

dashboard.json not syncing to docs/

The dual-write in `save.py` checks `if SITE_DIR.exists()`. Ensure `docs/` directory exists. If out of sync, copy manually:

```
cp data/dashboard.json docs/dashboard.json
```

Stale lock file

If a subagent crashes mid-write, `data/.dashboard.lock` may exist but isn't held. The `fcntl.flock` pattern is safe — stale lock files don't block (the lock is on the file descriptor, not the file's existence). Simply delete if concerned:

```
rm data/.dashboard.lock
```

Markdown parse errors in bootstrap

`python -m bd.dashboard` bootstraps from Markdown files. If parsing fails, check that report format matches expected patterns (e.g., `### CompanyName — Score NN`).

Subagent permission fallbacks

If subagents can't run Python (no Bash permission), the main conversation handles all work directly. Research is done in batches of 2-3; outreach can be done in a single large Python script since it doesn't require additional web research.

GitHub Pages not updating

- Confirm push reached `main` branch
- Check GitHub repo Settings → Pages → Source is set to `docs/` on `main`
- GitHub Pages can take 1-2 minutes to propagate

Appendix A — Module API Reference

bd/models.py

All Pydantic models (see Section 6 for full definitions).

bd/config.py

Constants only — no functions.

bd/formatting.py

Function	Signature	Returns
`format_revenue`	`(rev: float | None) -> str`	`"$2.0B"`, `"$500M"`, `"Unknown"`
`format_employees`	`(count: int | None) -> str`	`"5K+"`, `"350"`, `"Unknown"`

bd/discover/scorer.py

Function	Signature	Returns
`score_revenue`	`(prospect: Prospect) -> float`	0–25
`score_employees`	`(prospect: Prospect) -> float`	0–20
`score_signals`	`(prospect: Prospect) -> float`	0–35
`score_fit`	`(prospect: Prospect) -> float`	0–20
`score_prospect`	`(prospect: Prospect) -> Prospect`	Mutates `.score`, returns prospect

bd/discover/report.py

Function	Signature	Returns
`generate_report`	`(prospects: list[Prospect]) -> str`	Markdown report; also saves

bd/research/report.py

Function	Signature	Returns
`generate_dossier_report`	`(dossier: Dossier) -> str`	Markdown dossier; also saves

bd/outreach/drafter.py

Function	Signature	Returns
`generate_outreach_report`	`(package: OutreachPackage) -> str`	Markdown outreach; also saves

bd/market/report.py

Function	Signature	Returns
`generate_market_report`	`(sectors: list[MarketSector]) -> str`	Markdown report; also saves (replace)

bd/save.py

Function	Signature	Description
`save_discovery`	`(report: str, prospects: list[Prospect] | None) -> Path`	Append to discovery.md + JSON
`save_research`	`(report: str, dossier: Dossier | None) -> Path`	Append to research.md + JSON
`save_outreach`	`(report: str, package: OutreachPackage | None) -> Path`	Append to outreach.md + JSON
`save_market_intelligence`	`(report: str, sectors: list[MarketSector] | None) -> Path`	Replace market_intelligence.md + JSON
`clear_outreach`	`() -> None`	Reset outreach data
`clear_market_intelligence`	`() -> None`	Reset market data

bd/pipeline.py

Function	Signature	Description
`get_existing_prospects`	`() -> list[Prospect]`	Deduped prospects, highest score wins
`pipeline_status`	`() -> dict`	Counts + missing_dossiers/missing_outreach
`clear_phase`	`(phase: str) -> None`	Reset a pipeline phase

bd/dashboard.py

Function	Signature	Description
`export_dashboard_json`	`(prospects, dossiers, outreach_packages) -> Path`	Merge-export to JSON
`parse_discovery_markdown`	`(md: str) -> list[list[Prospect]]`	Parse discovery.md
`parse_research_markdown`	`(md: str) -> list[Dossier]`	Parse research.md
`parse_outreach_markdown`	`(md: str) -> list[OutreachPackage]`	Parse outreach.md
`bootstrap_from_markdown`	`() -> Path`	Rebuild dashboard.json from Markdown

populate_markets.py

CLI tool. `--status` prints freshness table. No `--status` runs the full seed + save.

Appendix B — Dashboard app.js Function Index

Function	Line	Purpose
`setupTabs`	13	Nav tab click handlers + hamburger toggle
`switchTab`	31	Activate a view, deactivate others
`setupScrollTop`	44	Show/hide scroll-to-top button
`loadData`	55	Fetch dashboard.json → renderAll()
`renderAll`	82	Call all view renderers
`formatRevenue`	96	Format revenue for display
`formatEmployees`	103	Format employee count for display

Function	Line	Purpose
`scoreClass`	109	CSS class based on score (green/amber/gray)
`signalLabel`	115	Human-readable signal type labels
`tierLabel`	127	Tier enum → display label
`tierClass`	136	Tier enum → CSS class
`escapeHtml`	143	XSS prevention
`categoryOf`	160	Industry string → display category
`renderFilterBar`	173	Create filter pill bar for a view
`groupByCategory`	195	Group items by industry category
`renderHome`	209	Home view: KPIs, capabilities, ICP, scoring
`renderPipeline`	442	Pipeline view: prospect cards by category
`renderPipelineCards`	478	Render prospect cards for a category filter
`prospectCard`	503	Single prospect card HTML
`signalItem`	544	Single signal badge HTML
`renderResearch`	559	Research view: dossier cards by category
`renderResearchCards`	587	Render dossier cards for a category filter
`dossierCard`	605	Single dossier card HTML
`orgSnapshotSection`	664	Dossier section 1 renderer
`companyOverviewSection`	678	Dossier section 1b renderer
`contactsSection`	700	Dossier section 3 renderer
`cultureSection`	737	Dossier section 4 renderer
`triggerEventsSection`	749	Dossier section 5 renderer

Function	Line	Purpose
`fitAssessmentSection`	768	Dossier section 6 renderer
`conversationSection`	802	Dossier section 7 renderer
`painPointsSection`	823	Legacy pain points renderer
`competitorsSection`	833	Legacy competitors renderer
`textSection`	843	Generic text section renderer
`richTextSection`	852	Markdown-to-HTML section renderer
`renderOutreach`	905	Outreach view: packages by category
`renderOutreachCards`	936	Render outreach cards for a category filter
`outreachCard`	955	Single outreach package card HTML
`linkedInCard`	995	LinkedIn message display
`coldEmailCard`	1011	Cold email version display with copy button
`legacyOutreachCard`	1028	Legacy outreach sequence card
`legacyEmailCard`	1068	Legacy email display
`freshnessBadge`	1094	Green/amber/gray freshness indicator
`renderMarketsSidebar`	1104	Markets sidebar: 10 most recent articles
`renderMarkets`	1164	Full markets page: sectors by category
`renderMarketsPageCards`	1202	Render market sector cards for a filter
`renderMarketCards`	1225	Render market cards (sidebar variant)
`marketCard`	1248	Single market sector card HTML
`articleItem`	1301	Single article list item HTML

Function	Line	Purpose
`attachMarketCardToggle`	1312	Expand/collapse for market cards
`setupMarketsSidebar`	1326	Sidebar open/close/overlay handlers
`renderProposals`	1368	Proposals view (coming soon)
`renderHowItWorks`	1462	How It Works view: full methodology
`renderFuture`	1976	Future roadmap view
`attachCardToggle`	2118	Generic card expand/collapse + copy buttons
`showToast`	2160	Toast notification for copy actions
`updateMeta`	2167	Update page title/meta with data counts